

What I set up for you today

A plain-English account of every step.

Project: ml-factor-investing-2026 | Date: 21 May 2026 | Branch: personb-models

The 30-second version

Your laptop now has everything it needs to run the project. I installed the tools, copied the stock-price data into the project folder, and wrote two new code files for you to build on. All of it has been committed to your branch.

There are a few real gaps that you and Bowen need to talk about (end of this report). Nothing is broken; some pieces of the original plan need a small redesign because the data we have is monthly, but the original plan assumed daily.

Words that show up below

Feature. A number computed for each stock each month that the model uses as input. E.g. "last 12 months" return" is a feature.

Model. A machine-learning recipe that takes features in and predicts a stock's next-month return.

Backtest. A simulation: pretend it is the past, let the model trade, see if it would have made money.

Sector-relative. Compare a stock only against others in the same industry (tech vs tech, banks vs banks), not against the whole market.

Branch. A version of the code separate from the team's main version. Yours is called **personb-models**. You commit there, then later open a pull request to merge into main.

Part 1 - Setting up your laptop

Before today, your laptop could not run the project. I made it so it can.

1.1 The Python problem

Your default Python is version 3.13. PyTorch (the deep-learning library this project uses for the neural-network model) does not yet have a Mac version that works with Python 3.13. So nothing would install.

Fix: I built a separate, project-only Python 3.12 environment inside a folder called **.venv** in the project. From now on, always run Python through this environment using:

```
$ .venv/bin/python <your-script.py>
```

(Not just "python", or you will use the system 3.13 which is missing all the packages.)

1.2 Three small library conflicts I fixed

When I ran *pip install -r requirements.txt* for the first time, three things broke. They are all known macOS issues, none are your fault. I fixed them quietly inside the project so you do not have to think about them:

- **numpy 2.x is too new for the PyTorch we have** - I pinned an older numpy (1.26).
- **The SHAP library expected the newer numpy** - I pinned an older SHAP (0.46) to match.
- **Two copies of a low-level library called "libomp"** get loaded into memory at the same time (one from Anaconda, one from the PyTorch package). They fight each other and the neural network hangs forever. I added a single line at the top of **models.py** that tells the system "it is OK for both to be loaded".

Net result: every required library imports cleanly. You can verify by running:

```
$ .venv/bin/python -c "import torch, xgboost, sklearn, pandas, hmmlearn, yfinance; print('ok')"
```

Part 2 - Getting the data in place

The project needs three kinds of data: stock prices, S&P 500 membership history, and macro indicators (VIX, interest rates). Bowen wrote the code that loads all three. I ran it.

2.1 The big CRSP file

You had four identical copies of a 494 MB stock-price file on your Desktop. I copied one of them into the project at **data/raw/CRSPData_1925_2022.csv**. The other three copies on your Desktop are safe to delete whenever you want.

2.2 The processed caches

Reading and cleaning a 494 MB CSV every time you run the project would be painfully slow (about a minute). So Bowen's code reads it once and writes a much faster format called **parquet** into **data/processed/**. After today, these files exist on your laptop:

File in data/processed/	Size	What it is
crsp_monthly.parquet	152 MB	Monthly U.S. stock prices, 1925-2022 (the fast version of the 494 MB CSV)
yfinance_monthly.parquet	0.7 MB	Monthly prices for 2023-2024 (CRSP ends in 2022; Bowen patches the gap)
returns_spliced_2019_2024.parquet	0.7 MB	The two sources above stitched together into one tidy 72-month x 615-tick

These cache files are *not* committed to git (they are too large, and they can be regenerated from the original CSV at any time). If you ever delete the data/processed/ folder, just re-run Bowen's loader script and it will rebuild itself.

Part 3 - The two files I wrote for you

These are the two files you own as Person B. They were empty stubs before today; now they contain working code.

3.1 src/factors.py

This file's job: turn raw monthly returns into *features* the model can learn from. It also tells each stock what industry (sector) it is in, so we can compare stocks fairly within their sector.

The main function you will call is:

```
build_feature_panel(start="2005-01-01", end="2024-12-31")  
-> a tidy table of features, one row per (stock, month)
```

Inside, it computes these features:

- **mom** - 12-month momentum (the return over the past year, ignoring the most recent month).
- **rev** - short-term reversal (last month's return; stocks that dropped tend to bounce back).
- **mvol** - monthly volatility (how jumpy the stock has been over the past 6 months).
- **ivol** - idiosyncratic volatility (how jumpy it is AFTER you remove what the whole market did).
- **log_mktcap** - the (log of) market value, i.e. how big the company is. Smaller companies tend to behave differently from giants.

After it computes each feature, it converts the raw number into a **sector-relative rank** between 0 and 1: a value of 0.9 means the stock is in the top 10% of its sector for that feature, that month. This is the "Layer 1" trick the framework document calls for.

3.2 src/models.py

This file's job: three different machine-learning recipes that learn from the features and predict each stock's next-month return.

- **LassoModel** - a simple linear recipe. Your baseline.
- **XGBoostModel** - a more sophisticated recipe based on decision trees. This is the primary model the framework document recommends.
- **NNModel** - a small neural network. The secondary recipe; interesting to compare against XGBoost.

All three speak the same simple language that Bowen's backtest engine expects:

```
model.fit(features, target) # learn the pattern  
model.predict(new_features) # apply what was learned
```

This means you can swap any of the three models in and out of the backtest without changing anything else.

Part 4 - Proof that everything works together

I did not just write code; I ran it. Here is what came back:

4.1 factors.py produced a real feature table

- Shape: 40,186 rows x 5 columns (one row per stock-month from 2019 to 2024)
- 615 unique stocks covered
- All features successfully ranked within sector
- Sector counts at the first date: Financials 92, IT 82, Industrials 80, etc. - looks like a normal S&P 500 distribution.

4.2 All three models trained and predicted

On a small synthetic test where the right answer is a known linear formula, all three recipes recovered most of the signal:

- Lasso: 0.585 correlation with the truth
- XGBoost: 0.632 correlation with the truth
- Neural network: 0.572 correlation with the truth

4.3 The full pipeline runs end-to-end

Features → XGBoost → portfolio → backtest, all wired together, no errors, finishes in 13 seconds. The backtest's Sharpe ratio (a measure of how good the strategy is) came out at -0.44.

Why is the Sharpe negative? Because the training data we have today only covers 2019-2024, so the model only sees 24 months of history before it has to make predictions. That is not enough. Once we extend the data back to 2005 (one of your next tasks), the number will tell us something real. Today's number just confirms the plumbing is connected.

Part 5 - What is unfinished, and why

Some pieces of the original plan cannot be done yet. None of these are blockers; they are decisions you and Bowen need to make.

5.1 Three of the eight features need daily data

The feature spec Bowen wrote assumed we would have *daily* stock prices. The data we actually have is *monthly*. So three of the eight features (dollar volume, daily volatility, daily idiosyncratic volatility) cannot be computed exactly as written.

What I did: I left them as *NotImplementedError* stubs so it is obvious they are missing, and I built monthly substitutes for the volatility ones. These substitutes are defensible in the final report.

5.2 Two features need data we do not have

Features 7 (Book-to-Market) and 8 (Earnings/Price) need a different dataset called *Compustat* that the TA was supposed to share. The DECISIONS.md file says the deadline for the TA to reply was yesterday (20 May). Ask Bowen what came of that.

If Compustat does not arrive, the project just uses 6 features instead of 8. That is fine - the academic paper this project is based on actually says price-based features are the most important.

5.3 Sector labels are approximate

There is no clean source of "what sector was every stock in on every historical date". I built a workable substitute: I use the current sector classification from the team's S&P 500 list, and for stocks that no longer exist (delisted, merged) I fall back to their industry code. 11 of 615 stocks ended up labelled "Unknown" - small enough to ignore.

5.4 The training data only covers 2019-2024

The framework wants us to train on 2005-2015, validate on 2016-2018, and test on 2019-2024. But the frozen returns table I used today only covers 2019-2024. To get the full history, you just need to re-run Bowen's freeze script with **start="2005-01-01"** instead of **start="2019-01-01"**. Easy change, just takes a few minutes of compute. I can do this with you in the next session.

Part 6 - What you should do next

In the order I would tackle them:

#	What	Why
1	Push your branch to GitHub (click "Publish Branch" on VS Code) and open a pull request so Bowen and Person C can review it.	Get VS Code and GitHub working so Bowen and Person C can review it.
2	Re-run the freeze script to cover 2005-2024 (not just 2019-2024) to get enough training history for a real backtest.	Get VS Code and GitHub working so Bowen and Person C can review it.
3	Talk to Bowen about the daily-vs-monthly question and the Compustat question (Part 5 says you can skip).	Get VS Code and GitHub working so Bowen and Person C can review it.
4	Add evaluation metrics: out-of-sample R-squared, and required for the final report for comparing two models. Both are in the README.	Get VS Code and GitHub working so Bowen and Person C can review it.
5	Tune XGBoost hyper-parameters on the 2016-2018 training data using Optuna (already installed).	Get VS Code and GitHub working so Bowen and Person C can review it.

Part 7 - What to say to Bowen

Four questions, in plain language:

Q1. About daily vs monthly data

"Three of the eight features in your spec need daily prices. We only have monthly. Are you willing to build a daily-price loader, or should we officially swap in monthly substitutes (which I have already implemented) and log it in DECISIONS.md?"

Q2. About Compustat

"What happened with the TA's reply about Compustat? The deadline you set in DECISIONS.md was 20 May. If we did not get access, can we agree to drop B/M and E/P and ship with 6 features?"

Q3. About the requirements.txt file

"On macOS with Anaconda Python 3.12, three things break the install. I worked around them inside my branch, but they will hit anyone setting up a fresh machine. Can we add these three lines to requirements.txt to spare future-us the pain?"

```
numpy<2
shap<0.47
# README note: if NN hangs on Mac, set KMP_DUPLICATE_LIB_OK=TRUE
```

Q4. About the backtest engine

"Your engine refits the model at every monthly rebalance, but the docstring describes a block-refit (refit only every *test_window* months). One of those two is what you intended; which one? It affects how slow the backtest runs."

Part 8 - Where everything was saved

Everything I changed is committed to your branch **personb-models**, in three separate commits so the history is easy to read:

Commit	What it contains
personb: scaffold factors.py and models.py	The two new code files. About 970 lines of new code.
docs: add reference materials	The 11 PDFs and Word documents from the documents/ folder (framework)
personb: add setup session status report	This PDF you are reading right now, plus the script that generates it.

Your **personb-models** branch is now 3 commits ahead of main. Click "Publish Branch" in VS Code to push it to GitHub, then open a pull request.

If something looks wrong later

- The two source files are at **src/factors.py** and **src/models.py**. Open them in VS Code; the comments inside explain each function.
- If a model hangs on your Mac, the first thing to check is that the line **os.environ.setdefault("KMP_DUPLICATE_LIB_OK", "TRUE")** at the top of **models.py** is still there.
- If you ever need to rebuild the data caches from scratch, run **.venv/bin/python -m src.data_loader** from the project folder and it will regenerate everything.
- If you want to read this report again later, it is at **PERSONB_SETUP_REPORT.pdf** in the project folder.